

Ascon-AEAD128

Cơ sở lý thuyết cho đồ án
thiết kế lõi IP mật mã tích hợp AMBA APB

Chuẩn tham chiếu: NIST SP 800-232

Tài liệu tự học — có từ điển thuật ngữ và hình minh họa

Mục lục và cách đọc

- Phần 0** Từ điển thuật ngữ
- Phần 1** Bối cảnh — từ mật mã hạng nhẹ tới đề tài đồ án
- Phần 2** Vì sao cần AEAD
- Phần 3** Mô hình sponge — trực giác cốt lõi
- Phần 4** Trạng thái và hàm hoán vị
- Phần 5** Quy tắc đệm và chia khối
- Phần 6** Bốn giai đoạn mã hóa
- Phần 7** Giải mã và kiểm tra tag
- Phần 8** Quy tắc bảo mật khi sử dụng
- Phần 9** Mười lỗi hay gặp nhất
- Phần 10** Câu hỏi tự kiểm tra
- Phụ lục** Tra cứu nhanh

Trình tự đọc

Tài liệu xây theo hình phễu — mỗi phần dựa trên phần trước, nên đọc tuần tự sẽ nhẹ nhất:

Nhóm	Phần	Trả lời câu hỏi
Chuẩn bị	0	Các từ chuyên môn nghĩa là gì
Đặt vấn đề	1 – 2	Vì sao có Ascon, vì sao cần AEAD
Nguyên lý	3 – 4	Thuật toán hoạt động dựa trên cơ chế gì
Chi tiết	5 – 7	Từng bước cụ thể làm gì
Vận dụng	8 – 10	Dùng sao cho đúng, tránh lỗi nào

Quy ước trình bày. Mỗi phần mở đầu bằng một dòng nêu *phần này trả lời câu hỏi gì*, và kết thúc bằng khung **Tóm lại** gồm vài ý cốt lõi. Thuật ngữ xuất hiện lần đầu được **in đậm màu** kèm giải thích ngắn ngay tại chỗ; toàn bộ thuật ngữ gom lại ở Phần 0.

Phần 0 — Từ điển thuật ngữ

Phần này trả lời: các từ chuyên môn xuất hiện trong tài liệu nghĩa là gì.

Không cần học thuộc. Đọc lướt một lượt để quen mặt chữ, rồi quay lại tra khi gặp.

0.1. Thuật ngữ mật mã chung

Thuật ngữ	Giải thích
Mật mã hạng nhẹ lightweight cryptography	Nhánh mật mã thiết kế riêng cho thiết bị ít tài nguyên — ít cổng logic, ít bộ nhớ, chạy pin. Đánh đổi: chấp nhận độ an toàn vừa đủ để lấy chi phí thấp.
Bản rõ — plaintext (P)	Dữ liệu gốc chưa mã hóa, thứ ta muốn giấu.
Bản mã — ciphertext (C)	Dữ liệu sau khi mã hóa. Không có khóa thì không đọc ra được gì.
Khóa — key (K)	Chuỗi bit bí mật mà chỉ bên gửi và bên nhận biết. Ascon-AEAD128 dùng khóa 128 bit.
Nonce (N)	Viết tắt của "number used once". Chuỗi bit công khai nhưng phải khác nhau mỗi lần mã hóa dưới cùng một khóa. Nó khiến hai lần mã hóa cùng nội dung vẫn cho ra bản mã khác nhau.
Tag — thẻ xác thực (T)	Chuỗi 128 bit gắn kèm gói tin, đóng vai trò "con dấu". Chỉ người có khóa mới tạo được. Sửa bất kỳ bit nào của gói tin thì tag kiểm tra ra sai.
AEAD	Authenticated Encryption with Associated Data — mã hóa có xác thực kèm dữ liệu liên kết. Một thuật toán làm cả ba việc: giấu, chống sửa, chống giả mạo.
Associated Data (A)	Dữ liệu cần <i>xác thực</i> nhưng <i>không cần giấu</i> , ví dụ header gói tin mà router phải đọc được.
Tính bí mật confidentiality	Người ngoài không đọc được nội dung.
Tính toàn vẹn integrity	Nếu ai đó sửa dữ liệu thì bên nhận phát hiện được.
Tính xác thực authenticity	Bên nhận chắc chắn gói tin do người nắm khóa tạo ra, không phải kẻ giả mạo.
Va chạm — collision	Hai đầu vào khác nhau cho ra cùng một kết quả. Với hàm băm, tìm được va chạm là phá được thuật toán.
Tấn công vét cạn brute force	Thử toàn bộ khả năng cho tới khi trúng. Khóa 128 bit nghĩa là phải thử 2^{128} lần — bất khả thi.
Tấn công kênh bên side-channel attack	Không phá toán học, mà đo <i>vật lý</i> : điện năng tiêu thụ, thời gian chạy, bức xạ điện từ của con chip để suy ra khóa.
Mặt nạ — masking	Biện pháp chống tấn công kênh bên: tách mỗi giá trị bí mật thành nhiều mảnh ngẫu nhiên, xử lý riêng rồi ghép lại, khiến điện năng tiêu thụ không còn liên quan tới dữ liệu thật.
Constant-time	Thuật toán chạy hết đúng bằng ngần ấy thời gian bất kể dữ liệu là gì. Nếu thời gian thay đổi theo dữ liệu, kẻ tấn công bấm giờ là suy ra được thông tin.
Tấn công phát lại replay attack	Kẻ tấn công không sửa gì, chỉ ghi lại một gói tin hợp lệ rồi gửi lại y nguyên vào lúc khác.
Test vector / KAT	Known Answer Test — bộ dữ liệu mẫu do NIST công bố, gồm khóa, nonce, bản rõ và bản mã đúng. Dùng để kiểm tra hiện thực của mình có chính xác không.

0.2. Thuật ngữ riêng của Ascon

Thuật ngữ	Giải thích
Sponge — mô hình bọt biển	Cách xây thuật toán mật mã: có một thanh ghi lớn, ta XOR dữ liệu vào một phần của nó rồi "khuấy" đều, lặp đi lặp lại. Giống nhúng bọt biển vào nước rồi vắt ra.
Trạng thái — state (S)	Thanh ghi 320 bit chứa toàn bộ thông tin thuật toán đang xử lý. Mọi thao tác đều diễn ra trên nó.
Từ — word	Đơn vị 64 bit. Trạng thái 320 bit chia thành 5 từ, ký hiệu S0 đến S4.
Rate (r)	Phần "hở" của trạng thái, nơi dữ liệu đi vào và bản mã đi ra. Với Ascon-AEAD128 là 128 bit đầu (S0, S1).
Capacity (c)	Phần "kín" của trạng thái, không bao giờ lộ ra ngoài. Với Ascon-AEAD128 là 192 bit còn lại (S2, S3, S4). Độ an toàn phụ thuộc phần này.
Hoán vị — permutation (p)	Hàm "khuấy trộn" 320 bit trạng thái. Là phép biến đổi một-một: mỗi đầu vào cho đúng một đầu ra, không mất thông tin.
Vòng — round	Một lần chạy qua ba lớp biến đổi. <code>p12</code> nghĩa là chạy 12 vòng, <code>p8</code> là 8 vòng.
Hộp thế — S-box	Bảng thay thế nhỏ, đổi một nhóm bit này thành nhóm bit khác. Đây là thành phần phi tuyến — thứ khiến thuật toán không giải được bằng đại số tuyến tính.
Phi tuyến / tuyến tính	Phép tuyến tính chỉ gồm XOR và dịch bit — có thể lập hệ phương trình để giải. Phép phi tuyến có nhân bit với nhau (AND), phá vỡ khả năng đó.
Bậc đại số algebraic degree	Số biến nhiều nhất nhân với nhau trong một số hạng. Hộp thế của Ascon có bậc 2 — rất thấp, nên rẻ để hiện thực và dễ áp mặt nạ.
Hiệu ứng thác avalanche effect	Đổi một bit đầu vào làm khoảng một nửa số bit đầu ra thay đổi. Đây là tính chất một thuật toán tốt phải có.
Khối — block	Dữ liệu được cắt thành từng đoạn bằng nhau để xử lý. Với Ascon-AEAD128, một khối là 128 bit = 16 byte.
Đệm — padding	Thêm bit vào cuối dữ liệu cho đủ một khối trọn vẹn, theo quy tắc cố định để bên nhận biết đâu là dữ liệu thật.
Phân tách miền domain separation	Kỹ thuật làm cho hai ngữ cảnh khác nhau không bao giờ cho ra cùng trạng thái, bằng cách chèn thêm một dấu hiệu phân biệt.
IV — initial value	Giá trị 64 bit cố định nạp vào đầu trạng thái, mã hóa các tham số của thuật toán (số vòng, độ dài tag, rate).
XOR (ký hiệu ⊕)	Phép "hoặc loại trừ": hai bit giống nhau cho 0, khác nhau cho 1. Tính chất quan trọng: <code>a ⊕ b ⊕ b = a</code> , nên XOR hai lần là quay về giá trị cũ.
Xoay vòng phải rotate right (>>>)	Dịch các bit sang phải, bit rơi khỏi đầu này thì chui vào đầu kia. Khác với dịch thường — không mất bit nào.
Little-endian	Quy ước xếp byte: byte có trọng số thấp nhất đứng trước. Chuẩn NIST dùng quy ước này. Xếp sai thứ tự là sai toàn bộ kết quả.
Online / single-pass	Online: xuất được bản mã ngay khi có dữ liệu, không cần biết trước tổng độ dài. Single-pass: chỉ duyệt dữ liệu một lần duy nhất.

0.3. Thuật ngữ thiết kế phần cứng

Thuật ngữ	Giải thích
Lõi IP — IP core	Intellectual Property core. Một khối mạch đã thiết kế sẵn, đóng gói kèm giao diện chuẩn, cắm được vào nhiều hệ thống khác nhau.
RTL	Register Transfer Level. Mức mô tả mạch bằng thanh ghi và logic giữa chúng, viết bằng Verilog hoặc VHDL.
FPGA / ASIC	FPGA là chip lập trình lại được, dùng để thử nghiệm. ASIC là chip chế tạo cố định cho một mục đích, rẻ và nhanh hơn khi sản xuất số lượng lớn.
AMBA	Advanced Microcontroller Bus Architecture — bộ chuẩn bus của ARM, gồm AXI (nhanh), AHB (trung bình) và APB (đơn giản).
APB	Advanced Peripheral Bus. Bus đơn giản nhất họ AMBA, dùng cho ngoại vi băng thông thấp. Mỗi giao dịch gồm hai pha: SETUP rồi ACCESS.
Master / Slave	Master là bên chủ động phát lệnh (thường là vi xử lý). Slave là bên bị động đáp ứng. Lõi IP của đồ án đóng vai slave.
Máy trạng thái FSM	Finite State Machine. Mạch điều khiển ghi nhớ mình đang ở bước nào và quyết định bước tiếp theo.
Thông lượng throughput	Số bit xử lý được mỗi giây.
Độ trễ — latency	Thời gian từ lúc đưa dữ liệu vào tới lúc có kết quả.
Fmax	Tần số xung nhịp cao nhất mạch còn chạy đúng.
PPA	Power, Performance, Area — ba chỉ tiêu đánh giá một thiết kế: điện năng, hiệu năng, diện tích.
LFSR	Linear Feedback Shift Register. Mạch sinh chuỗi số gần ngẫu nhiên rất rẻ, hợp để tạo nonce không trùng lặp.

Phần 1 — Bối cảnh và đề tài

Phần này trả lời: vì sao có Ascon, vì sao đề án chọn biến thể AEAD128, và vì sao ghép với bus APB.

1.1. Bài toán xuất phát

Số lượng thiết bị nhúng kết nối mạng đang tăng rất nhanh: cảm biến công nghiệp, thẻ RFID, thiết bị đo thông minh, thiết bị y tế đeo trên người. Chúng có một đặc điểm chung — **hạn chế tài nguyên nghiêm trọng**. Diện tích chip tính bằng vài nghìn cổng logic, bộ nhớ tính bằng kilobyte, và nhiều thiết bị chạy pin nhiều năm không thay.

Những thiết bị này vẫn cần bảo mật. Nhưng các chuẩn mật mã phổ thông như AES-GCM hay SHA-2 được thiết kế cho môi trường máy chủ và máy tính cá nhân. Đưa chúng xuống thiết bị siêu nhỏ thì tốn quá nhiều diện tích, quá nhiều năng lượng, và hiệu suất rất kém với các gói tin ngắn — vốn là loại lưu lượng chủ yếu của IoT.

Mật mã hạng nhẹ là nhánh mật mã tối ưu hóa sự cân bằng giữa độ an toàn, hiệu năng và chi phí tài nguyên cho đúng lớp thiết bị này.

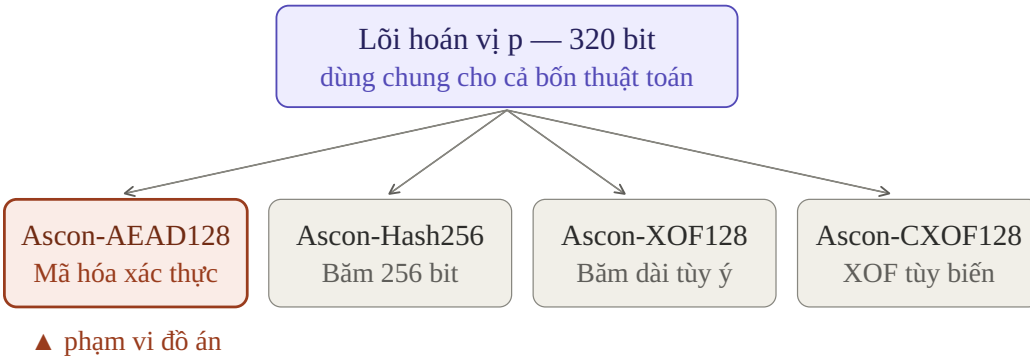
1.2. Quy trình chuẩn hóa của NIST

Thời gian	Sự kiện
2013	NIST bắt đầu nghiên cứu hiệu năng mật mã trên thiết bị hạn chế tài nguyên
8/2018	Công bố yêu cầu đệ trình chính thức cho quy trình chuẩn hóa
2/2023	Sau nhiều vòng đánh giá công khai, NIST chọn họ thuật toán Ascon
8/2025	Xuất bản tiêu chuẩn chính thức NIST SP 800-232

Ascon do nhóm Dobraunig, Eichlseder, Mendel và Schl  ffer thiết kế. Nó vượt qua 56 ứng viên khác nhờ ba điểm: hiệu năng tốt trên cả phần cứng lẫn phần mềm, cấu trúc toán học đơn giản dễ phân tích, và **khả năng chống tấn công kênh bên với chi phí thấp** — yếu tố then chốt với thiết bị vật lý mà kẻ tấn công có thể cầm trên tay.

1.3. Họ Ascon — một lõi, bốn thuật toán

Điểm thiết kế thông minh nhất của Ascon: cả bốn thuật toán trong chuẩn đều dùng **chung một lõi ho  n vị**. Chúng chỉ khác nhau ở lớp vỏ bọc bên ngoài.



H  nh 1 — Họ thuật toán Ascon theo NIST SP 800-232

Hệ quả trực tiếp cho thiết kế phần cứng: nếu sau này cần bổ sung chức năng băm, bạn **không phải thêm một khối SHA-256 riêng**. Chỉ cần vài bộ ghép kênh và vài trạng thái trong máy trạng thái, quanh chính lõi hoán vị đã có.

1.4. Vì sao đề án chọn Ascon-AEAD128

- Nó giải quyết trọn vẹn bài toán truyền tin an toàn. Ba thuật toán còn lại chỉ là hàm băm — bảo vệ tính toàn vẹn nhưng không mã hóa. Thiết bị IoT gửi số liệu cần cả bí mật lẫn xác thực.
- Nó là biến thể tốn công nhất để hiện thực — bốn giai đoạn, có khóa, nonce, tag, phân nhánh mã hóa/giải mã. Làm được nó thì ba cái còn lại gần như cho không.
- Nó có bộ test vector chính thức từ NIST, nên kết quả đề án kiểm chứng được khách quan.

1.5. Một lưu ý quan trọng về tên gọi

"**ASCON-128**" và "**Ascon-AEAD128**" là hai thuật toán khác nhau. Cùng khóa, cùng nonce, cùng bản rõ, chúng cho ra bản mã khác nhau hoàn toàn. Nhầm lẫn ở đây làm sai toàn bộ đề án.

	Ascon-128 (bản v1.2)	Ascon-AEAD128 (SP 800-232)
Nguồn gốc	Bản đề trình dự thi	Chuẩn chính thức, 8/2025
Rate	64 bit	128 bit
Hoán vị trung gian	p6	p8
Thứ tự byte	Big-endian	Little-endian
IV	0x80400c0600000000	0x00001000808c0001

Chuẩn NIST thực ra dựa trên biến thể **Ascon-128a** của bản v1.2 (rate 128, p8), rồi đổi sang little-endian và thay IV mới — chứ không phải dựa trên Ascon-128.

Tên đề tài đề án ghi "ASCON-128" theo cách gọi quen thuộc, nhưng nội dung thực hiện phải bám **chuẩn NIST SP 800-232**, vì đó là tiêu chuẩn còn hiệu lực và là bộ test vector dùng để nghiệm thu. Trong báo cáo nên có một mục ngắn nêu rõ sự phân biệt này cùng lý do lựa chọn.

1.6. Vì sao hiện thực bằng phần cứng

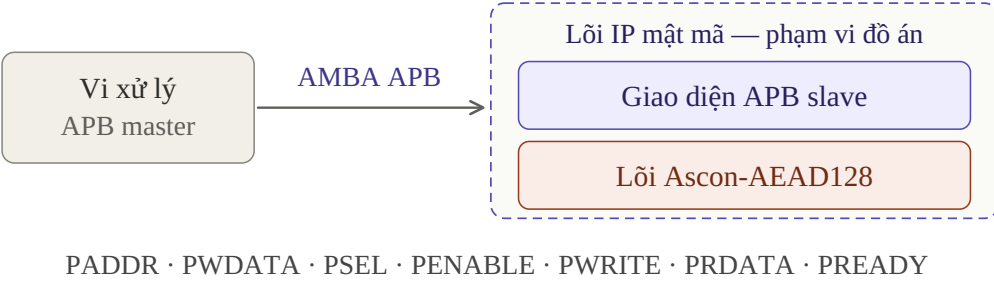
Ascon chạy bằng phần mềm trên vi điều khiển hoàn toàn được. Nhưng một **lỗi IP** phần cứng chuyên dụng có ba ưu thế mà phần mềm không thể có:

- Thông lượng cao hơn nhiều bậc.** Toàn bộ 320 bit trạng thái được xử lý song song mỗi chu kỳ, thay vì tuần tự qua thanh ghi 32 bit.
- Năng lượng trên mỗi bit thấp hơn hẳn** — yếu tố quyết định với thiết bị chạy pin.
- Giải phóng CPU.** Vi xử lý chỉ cần ghi dữ liệu vào thanh ghi rồi chờ chờ báo xong.

Ascon đặc biệt hợp với phần cứng vì nó **không dùng bảng tra, không có key schedule, không có phép nhân hay phép cộng số học**. Toàn bộ thuật toán chỉ gồm AND, XOR, NOT và dịch vòng dây — những thứ rẻ nhất trên silicon.

1.7. Vì sao ghép với AMBA APB

Một lõi mật mã không tự nó là sản phẩm. Nó phải nằm trong hệ thống, để vi xử lý nạp khóa, đẩy dữ liệu vào và đọc kết quả ra.



Hình 2 — Vị trí lõi IP trong hệ thống

APB được chọn vì nó là bus đơn giản nhất trong họ **AMBA**: không burst, không pipeline, mỗi giao dịch gồm hai pha SETUP và ACCESS. Đúng loại bus mà công nghiệp dùng cho ngoại vi băng thông thấp — và một lõi mật mã nhận vài chục byte mỗi lần chính là ngoại vi như vậy.

Với đồ án, sự đơn giản đó là ưu điểm kép: bạn tập trung công sức vào phần thuật toán thay vì vật lộn với giao thức bus, nhưng vẫn có được một lõi IP đúng chuẩn công nghiệp.

1.8. Phạm vi và giới hạn

Thực hiện	Không thực hiện
Ascon-AEAD128, cả mã hóa và giải mã	Ascon-Hash256, XOF128, CXOF128
Giao diện APB slave đầy đủ	Các bus khác (AHB, AXI)
Kiểm chứng bằng test vector NIST	Chống tấn công kênh bên bằng mặt nạ
Tổng hợp và báo cáo PPA	Cơ chế chống tấn công phát lại

Tóm lại phần 1

- Thiết bị IoT quá nhỏ để chạy AES, nên NIST chuẩn hóa mật mã hạng nhẹ và chọn Ascon.
- Họ Ascon có 4 thuật toán dùng chung một lõi hoán vị; đồ án làm biến thể AEAD128.
- Ascon-128 (v1.2) khác Ascon-AEAD128 (NIST) — phải bám chuẩn NIST.
- Làm phần cứng để có thông lượng cao và tiết kiệm năng lượng; ghép APB để lõi tái sử dụng được.

Phần 2 — Vì sao cần AEAD

Phần này trả lời: mã hóa thôi thiếu cái gì, và AEAD bù vào chỗ nào.

2.1. Mã hóa thôi là chưa đủ

Giả sử bạn có một cảm biến IoT gửi số liệu về server, và bạn mã hóa payload. Xong chưa? **Chưa**. Mã hóa chỉ giải quyết được việc *giấu nội dung*. Nó không ngăn được kẻ tấn công *sửa* nội dung.

Ví dụ cụ thể: với một số chế độ mã hóa dòng, $C = P \oplus \text{keystream}$. Kẻ tấn công không đọc được P , nhưng nếu hán lật bit thứ 5 của C thì bit thứ 5 của P cũng lật theo khi giải mã — nhờ tính chất $a \oplus b \oplus b = a$ của phép **XOR**. Hẳn không biết nội dung nhưng vẫn **sửa được nội dung**. Nhiệt độ 25 °C có thể biến thành 89 °C.

Bảo mật thật sự cần **ba** tính chất, không phải một:

Tính chất	Nghĩa	Không có thì sao
Tính bí mật	Người ngoài không đọc được	Lộ dữ liệu
Tính toàn vẹn	Sửa một bit là phát hiện được	Bị chỉnh sửa ngầm
Tính xác thực	Chỉ người có khóa tạo được gói hợp lệ	Bị giả mạo nguồn gửi

2.2. Cách cũ và vấn đề của nó

Cách truyền thống là ghép hai thuật toán: mã hóa bằng AES-CBC, rồi tính HMAC-SHA256 lên bản mã. Nghe hợp lý, nhưng có ba vấn đề:

- Phải chạy hai lượt** qua dữ liệu, tức là chậm gấp đôi.
- Phải quản lý hai khóa** và tự lo việc dẫn xuất khóa.
- Rất dễ ghép sai**. Ghép sai thứ tự đã từng gây lỗ hổng thật trong TLS — loại lỗi mà lập trình viên bình thường không có cách nào tự phát hiện.

Trên phần cứng nhỏ, vấn đề còn nặng hơn: phải tổng hợp **cả AES lẫn SHA-256** lên cùng một con chip. Diện tích tăng gấp đôi, mà thiết bị IoT thì đếm từng cổng logic.

2.3. AEAD gộp tất cả làm một

AEAD nghĩa là một thuật toán, một khóa, một lần duyệt dữ liệu, cho ra cả bản mã lẫn thẻ xác thực.

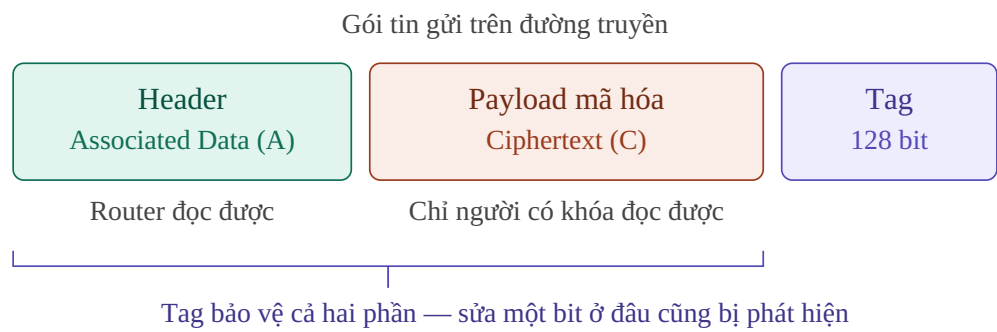
$\text{Enc}(K, N, A, P)$	$\rightarrow$	(C, T)
$\text{Dec}(K, N, A, C, T)$	$\rightarrow$	$P \quad \text{hoặc} \quad \perp$

Ký hiệu $\perp$ (đọc là "bottom") nghĩa là **từ chối** — không trả về gì cả.

Điểm cực kỳ quan trọng cho đồ án. Giải mã **không** trả về "bản rõ kèm cờ lỗi". Nó trả về **hoặc** bản rõ **hoặc** không gì cả. Nếu thiết kế của bạn đẩy bản rõ ra bus rồi mới báo tag sai, bạn đã vi phạm đặc tả — và đó là một lỗ hổng bảo mật thật.

2.4. Associated Data là gì

Là phần dữ liệu cần **xác thực nhưng không cần giấu**. Ví dụ điển hình trong IoT là header của một gói tin mạng.



Hình 3 — Cấu trúc một gói tin AEAD

Header **phải** để rõ vì router trung gian cần đọc để chuyển tiếp — nó không có khóa. Nhưng cũng **không được phép** cho kẻ tấn công sửa địa chỉ đích. Associated Data giải quyết đúng bài toán đó.

Tóm lại phần 2

- Mã hóa chỉ cho tính bí mật; còn thiếu tính toàn vẹn và tính xác thực.
- Ghép AES + HMAC được, nhưng chậm gấp đôi, tốn diện tích gấp đôi và dễ ghép sai.
- AEAD gộp cả ba tính chất vào một thuật toán, một khóa, một lượt duyệt.
- Associated Data là phần để rõ nhưng vẫn được tag bảo vệ.

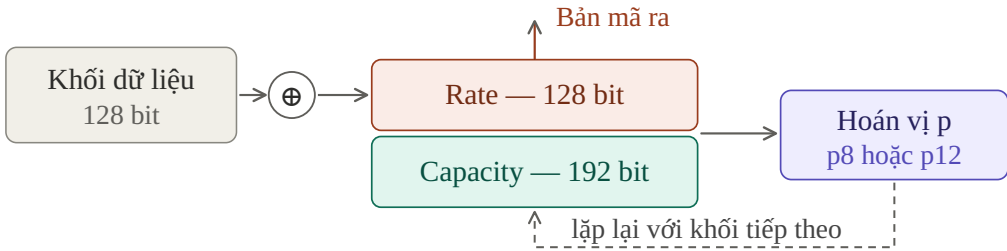
Phần 3 — Mô hình sponge

Phần này trả lời: cơ chế nền tảng mà cả bốn thuật toán Ascon dựa vào là gì.

3.1. Trực giác

Toàn bộ Ascon xoay quanh **một thanh ghi 320 bit**, chia làm hai ngăn, và chỉ có **hai động tác** lặp đi lặp lại:

- Hấp thụ** (absorb) — XOR một khối dữ liệu vào ngăn **rate**.
- Khuấy** (permute) — chạy hàm hoán vị **p** để trộn đều cả 320 bit.



Người ngoài chỉ thấy phần Rate. Phần Capacity không bao giờ lộ ra.

Hình 4 — Chu trình hấp thụ và khuấy của sponge

3.2. Vì sao an toàn

Kẻ tấn công thấy toàn bộ rate, vì bản mã chính là rate sau khi XOR. Nhưng **capacity** luôn bị giấu. Sau mỗi lần **p**, bit của capacity "rơi" ngược vào rate và ảnh hưởng tới mọi thứ tiếp theo. Muốn tấn công, hẳn phải **đảo ngược p** để suy ra capacity — mà **p** được thiết kế để không đảo ngược nổi trong thời gian hữu hạn.

3.3. Đánh đổi cốt lõi

r quyết định tốc độ. c quyết định độ an toàn. Và $r + c = 320$ là cố định. Không thể có cả hai — đây là lý do tồn tại của mọi biến thể Ascon.

Biến thể	r	c	Nhận xét
Ascon-AEAD128	128	192	Nhanh — mỗi lần khuấy nuốt 16 byte
Ascon-Hash256	64	256	Chậm hơn nhưng an toàn hơn

Hàm băm cần **c** lớn vì phải chống tấn công **va chạm**, vốn dễ hơn tấn công vét cạn. AEAD có khóa bảo vệ nên có thể lấy **r** lớn hơn để đổi lấy tốc độ.

3.4. Riêng AEAD có thêm một mẹo

Sponge thuần không có khóa. Ascon-AEAD128 thêm vào: **khóa K được nhét vào lúc bắt đầu và nhét lại lúc kết thúc**. Kết quả là không biết **K** thì không dựng lại được trạng thái ở cả hai đầu, nên không tạo được tag hợp lệ, nên không giả mạo được.

Tóm lại phần 3

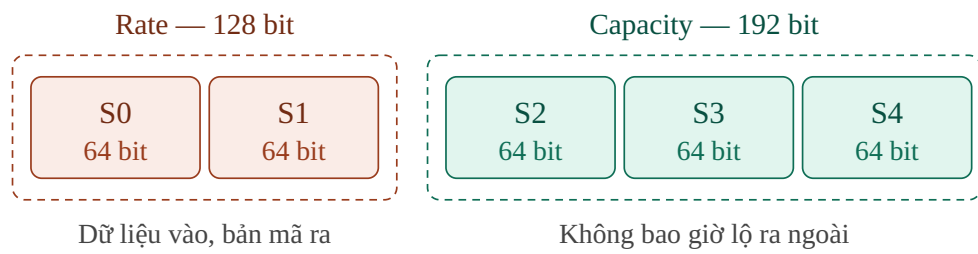
- Trạng thái 320 bit chia thành rate (hở) và capacity (kín).
- Chỉ có hai động tác: XOR dữ liệu vào rate, rồi khuấy toàn bộ trạng thái.
- Rate lớn thì nhanh, capacity lớn thì an toàn — tổng luôn bằng 320.
- AEAD thêm khóa ở hai đầu để chống giả mạo tag.

Phần 4 — Trạng thái và hàm hoán vị

Phần này trả lời: hàm khuấy p làm gì cụ thể, gồm những lớp nào.

4.1. Trạng thái 320 bit

Trạng thái chia thành 5 từ 64 bit: $s = s_0 \parallel s_1 \parallel s_2 \parallel s_3 \parallel s_4$ (ký hiệu $\parallel$ là nối chuỗi).



Hình 5 — Trạng thái 320 bit của Ascon-AEAD128

Cũng có thể xem trạng thái như ma trận 5 hàng $\times$ 64 cột. Cách nhìn này quan trọng vì lớp phi tuyến làm việc **theo cột** còn lớp tuyến tính làm việc **theo hàng**.

4.2. Một vòng hoán vị gồm ba lớp

$s \rightarrow [p_C: \text{ cộng hằng số }] \rightarrow [p_S: \text{ hộp thể }] \rightarrow [p_L: \text{ khuếch tán }] \rightarrow s'$
phá đối xứng phi tuyến lan truyền

Chạy 12 vòng gọi là **p12**, chạy 8 vòng gọi là **p8**. Cùng một cỗ máy, chỉ khác số lần quay.

Lớp p_C — cộng hằng số vòng

$s_2 = s_2 \oplus c_i$, chỉ tác động lên 8 bit thấp của S_2 vì hằng số có dạng $0x00000000000000XX$.

idx	c	idx	c	idx	c	idx	c
0	3c	4	f0	8	b4	12	78
1	2d	5	e1	9	a5	13	69
2	1e	6	d2	10	96	14	5a
3	0f	7	c3	11	87	15	4b

Quy tắc chọn: hoán vị rnd vòng, ở vòng thứ i , dùng $c[16 - rnd + i]$. Nghĩa là **p12** chạy index 4 $\rightarrow$ 15, còn **p8** chạy index 8 $\rightarrow$ 15.

Mẹo cho phần cứng. Cả p8 và p12 đều kết thúc ở index 15. Nói cách khác, p8 chính là 8 vòng cuối của p12. Bạn chỉ cần một bộ đếm chung chạy tới 15, không cần hai bảng hằng số.

Vì sao cần hằng số? Không có nó, mọi vòng giống hệt nhau. Trạng thái toàn 0 sẽ mãi là toàn 0, và các trạng thái đối xứng sẽ giữ nguyên tính đối xứng — cả hai đều là lỗ hổng.

Lớp p_S — hộp thế 5 bit

Đây là **phần phi tuyến duy nhất** của toàn thuật toán. Không có nó, cả Ascon chỉ là một hệ phương trình tuyến tính và giải được bằng đại số tuyến tính trong vài giây.

Áp dụng **64 hộp thế 5 bit song song**, mỗi cái lên một cột:

```
y0 = x4·x1 ⊕ x3 ⊕ x2·x1 ⊕ x2 ⊕ x1·x0 ⊕ x1 ⊕ x0
y1 = x4 ⊕ x3·x2 ⊕ x3·x1 ⊕ x3 ⊕ x2·x1 ⊕ x2 ⊕ x1 ⊕ x0
y2 = x4·x3 ⊕ x4 ⊕ x2 ⊕ x1 ⊕ 1
y3 = x4·x0 ⊕ x4 ⊕ x3·x0 ⊕ x3 ⊕ x2 ⊕ x1 ⊕ x0
y4 = x4·x1 ⊕ x4 ⊕ x3 ⊕ x1·x0 ⊕ x1
```

Dấu **·** là phép AND (nhân bit), dấu **⊕** là XOR.

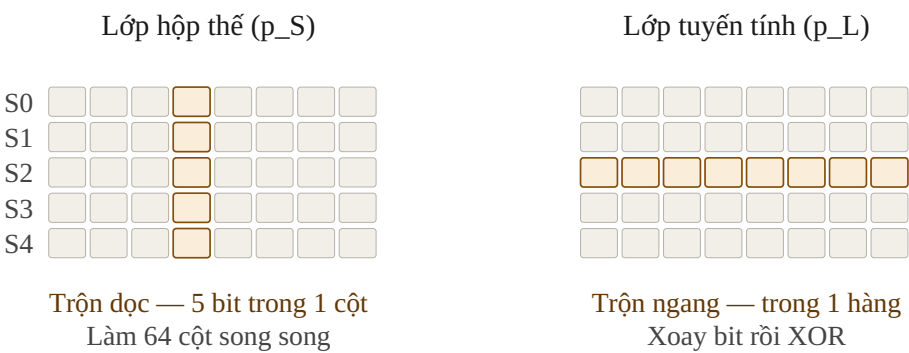
Tính chất đáng viết vào báo cáo. Mỗi số hạng nhiều nhất là tích của 2 biến — ta nói hộp thế có **bậc đại số bằng 2**. Hệ quả: cài đặt rất rẻ (chỉ AND và XOR, không cần bảng tra), và **rất dễ áp mật nạ chống tấn công kênh bên**. Đây là một trong những lý do chính NIST chọn Ascon.

Lớp p_L — khuếch tán tuyến tính

```
S0 = S0 ⊕ (S0 » 19) ⊕ (S0 » 28)
S1 = S1 ⊕ (S1 » 61) ⊕ (S1 » 39)
S2 = S2 ⊕ (S2 » 1) ⊕ (S2 » 6)
S3 = S3 ⊕ (S3 » 10) ⊕ (S3 » 17)
S4 = S4 ⊕ (S4 » 7) ⊕ (S4 » 41)
```

Ký hiệu **»** là **xoay vòng phải**. Trên phần cứng, xoay bit **không tốn một cổng logic nào** — nó chỉ là đổi thứ tự dây.

4.3. Vì sao cần cả hai lớp



Hình 6 — Hai chiều trộn vuông góc nhau

	Trộn theo chiều	Không làm được gì
p_S	Dọc — giữa 5 hàng, trong cùng một cột	Bit không sang được cột khác
p_L	Ngang — trong một hàng, giữa các cột	Bit không sang được hàng khác

Chỉ có một trong hai thì bit bị kẹt mãi trong cột hoặc hàng của nó. Xen kẽ hai lớp vài vòng thì **một bit đầu vào ảnh hưởng tới toàn bộ 320 bit** — đó là **hiệu ứng thác**. Sau 3–4 vòng là đã lan hết; 12 vòng là biên an toàn rất rộng.

Tóm lại phần 4

- Trạng thái là 5 từ 64 bit, xem được như ma trận $5 \text{ hàng} \times 64 \text{ cột}$.
- Một vòng gồm ba lớp: cộng hằng số, hộp thể phi tuyến, khuếch tán tuyến tính.
- Hộp thể trộn theo cột, lớp tuyến tính trộn theo hàng — bù nhau tạo hiệu ứng thác.
- p8 chính là 8 vòng cuối của p12, nên phần cứng dùng chung một bộ đếm.

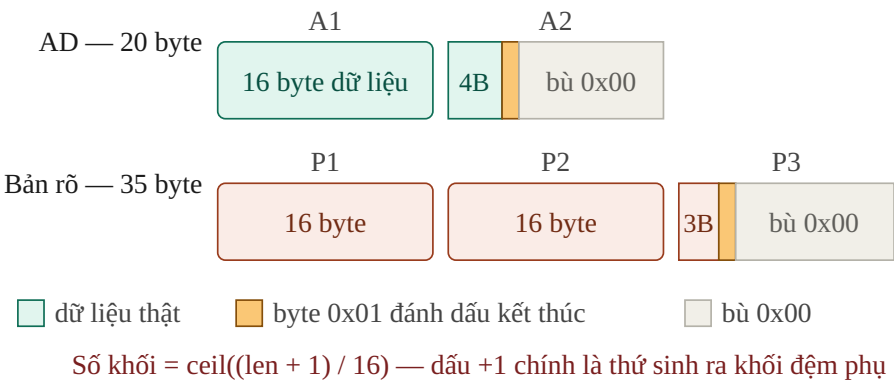
Phần 5 — Quy tắc đệm và chia khối

Phần này trả lời: dữ liệu dài ngắn tùy ý được cắt thành khối 16 byte như thế nào.

Phần này ngắn nhưng là nơi test vector fail nhiều nhất. Nếu bạn chỉ đọc kỹ một phần trong cả tài liệu, hãy chọn phần này.

5.1. Quy tắc

Dữ liệu vào phải chia thành các **khối** đúng bằng rate (16 byte với AEAD128). Nếu không vừa, phải **đệm**: thêm **một bit 1**, rồi thêm **bit 0** cho tới khi đủ khối. Trong biểu diễn **little-endian** của SP 800-232, ở mức byte điều này hiện ra là **chèn byte 0x01** ngay sau byte dữ liệu cuối, rồi bù 0x00.



Hình 7 — Chia khối và đệm cho AD 20 byte và bản rõ 35 byte

5.2. Hai cái bẫy

Bẫy 1 — Dữ liệu vừa khít khối vẫn phải thêm nguyên một khối đệm

Nếu AD dài đúng 16 byte, bạn **không** xử lý 1 khối mà xử lý **2 khối**: khối dữ liệu, và một khối đệm toàn bộ (01 00 00 ... 00).

Vì sao? Nếu không, hai thông điệp khác nhau sẽ cho ra cùng một dãy khối — mà thế là va chạm, là lỗ hổng. Quy tắc đệm **bắt buộc phải đảo ngược được**: nhìn vào dãy khối phải suy ra được duy nhất dữ liệu gốc.

Bẫy 2 — AD và bản rõ xử lý khác nhau khi rỗng

	AD rỗng	Bản rõ rỗng
Số khối	0 khối	1 khối (chỉ chứa đệm)
Có chạy hoán vị không	Không lần nào	Có, nhưng là khối cuối nên không chạy p8 sau nó
Đầu ra	—	C rỗng (bị cắt về 0 byte)

Nói cách khác: **AD rỗng thì bỏ qua hoàn toàn giai đoạn AD; bản rõ rỗng thì vẫn phải xử lý một khối đệm.** Nếu bạn code `while (còn dữ liệu)` cho cả hai thì sẽ sai cả hai trường hợp.

5.3. Bảng tự kiểm tra

Cho `r = 16 byte`. Tự tính rồi đối chiếu — nếu khớp cả 7 dòng thì bạn đã hiểu đúng phần đệm.

A	Khối AD	P	Khối bản rõ	Số lần p8	Tổng chu kỳ
0	0	0	1	0	24
0	0	16	2	1	32
5	1	5	1	1	32
15	1	20	2	2	40
16	2 ← bẫy	32	3 ← bẫy	4	56
20	2	35	3	4	56
100	7	200	13	19	176

Chu kỳ tính theo kiến trúc một vòng mỗi chu kỳ: 12 (khởi tạo) + 8×(số khối AD) + 8×(số khối bản rõ – 1) + 12 (hoàn tất).

Tóm lại phần 5

- Đệm là thêm byte `0x01` rồi bù `0x00` cho đủ 16 byte.
- Số khối = $\text{ceil}((\text{độ dài} + 1) / 16)$ — dữ liệu vừa khít vẫn sinh thêm một khối.
- AD rỗng thì bỏ qua hẵn; bản rõ rỗng thì vẫn xử lý một khối đệm.

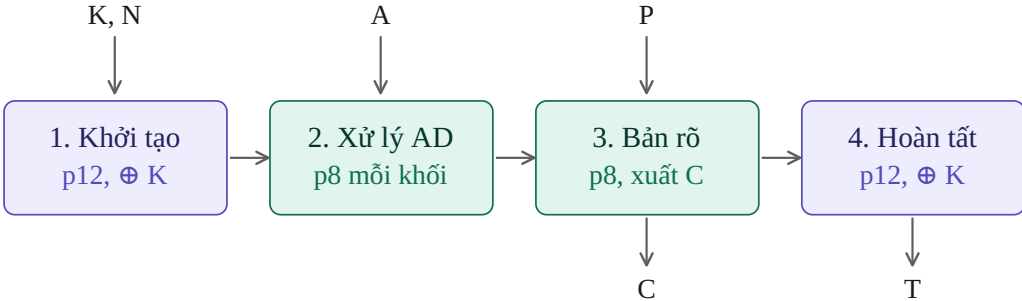
Phần 6 — Bốn giai đoạn mã hóa

Phần này trả lời: từ khóa và bản rõ, thuật toán đi qua những bước nào để ra bản mã và tag.

6.1. Tham số

Ký hiệu	Độ dài	Ghi chú
K — khóa	128 bit	bí mật
N — nonce	128 bit	công khai, bắt buộc duy nhất
A — associated data	0 ... tùy ý	không mã hóa
P — bản rõ	0 ... tùy ý	
C — bản mã	= P	không phình ra
T — tag	128 bit	
IV	64 bit	0x00001000808c0001
r / c	128 / 192	rate / capacity

6.2. Toàn cảnh



Tím = giai đoạn có XOR khóa · Xanh = giai đoạn hấp thụ dữ liệu

Hình 8 — Bốn giai đoạn của Ascon-AEAD128

Chú ý: **khóa K xuất hiện ở hai đầu, cách xa nhau**. Khúc giữa chạy tự do — đó chính là lý do khúc giữa dùng p8 thay vì p12.

6.3. Giai đoạn 1 — Khởi tạo

```
S0      = IV                      = 0x00001000808c0001
S1||S2  = K                      (128 bit khóa)
S3||S4  = N                      (128 bit nonce)

S = p12(S)

S3 ⊕= K[127:64]
S4 ⊕= K[63:0]
```

- **IV mã hóa chính tham số thuật toán** — mã định danh, số vòng, độ dài tag và rate. Nhờ vậy Ascon-AEAD128 và Ascon-Hash256 không bao giờ cho ra cùng trạng thái. Đây là **phân tách miền**.
- **Vì sao p12 chứ không p8?** Đây là điểm tiếp xúc trực tiếp với khóa, cần biên an toàn cao nhất.
- **Vì sao XOR K lần nữa sau p12?** Sau bước này, kẻ tấn công dù biết trạng thái cũng không quay ngược về trạng thái ban đầu để suy ra K, vì hẵn thiếu đúng phần K vừa được XOR vào.

6.4. Giai đoạn 2 — Associated Data

```
NẾU |A| > 0:
  chia pad(A) thành các khối 128-bit A1 ... Aa
  VỚI MỖI khối Ai:
    S0 ⊕= Ai[127:64]
    S1 ⊕= Ai[63:0]
    S = p8(S)                                ← chạy sau MỖI khối, kể cả khối cuối

# luôn thực hiện, kể cả khi A rỗng:
S4 ⊕= 0x8000000000000000                    ← bit phân tách miền
```

Về bit phân tách miền. Đây chỉ là *một bit* XOR vào cuối trạng thái, nhưng nó chống được một kiểu tấn công thật. Không có nó, kẻ tấn công có thể **dời ranh giới giữa A và P** mà trạng thái không đổi: gởi (A="abc", P="def") và (A="ab", P="cdef") sẽ cho ra cùng tag — một dạng giả mạo. Bit này đánh dấu rõ "hết AD, bắt đầu bản rõ". Nó **phải được XOR kể cả khi A rỗng**.

Vì sao p8 chứ không p12? Ở đây trạng thái đã bị khóa hóa từ giai đoạn 1, kẻ tấn công không kiểm soát được nó. 8 vòng là đủ biên an toàn, và tiết kiệm 33 % công sức.

6.5. Giai đoạn 3 — Bản rõ

```
chia pad(P) thành các khối 128-bit P1 ... Pm      (m ≥ 1 luôn luôn)

VỚI i = 1 ĐẾN m:
  S0 ⊕= Pi[127:64]
  S1 ⊕= Pi[63:0]
  Ci = S0 || S1                                ← đọc rate ra làm bản mã, NGAY
  NẾU i < m:  S = p8(S)                        ← KHÔNG chạy sau khối cuối

C = cắt(C1 || ... || Cm, về đúng |P| byte)
```

- **Không chạy p8 sau khối cuối.** Đây là bug kinh điển. Lý do: p12 của giai đoạn 4 sẽ lo việc trộn.
- **Bản mã chính là trạng thái rate.** Dữ liệu được hấp thụ vào và bản mã được vắt ra cùng một lúc, cùng một chỗ.
- **Cắt bản mã về đúng độ dài gốc**, nhờ vậy `|C| = |P|` chính xác.

Tính chất online: `Ci` có ngay khi có `Pi`, không cần biết tổng độ dài thông điệp. Với thiết kế phần cứng thì rất đáng giá — không cần bộ đệm chứa cả gói tin.

6.6. Giai đoạn 4 — Hoàn tất

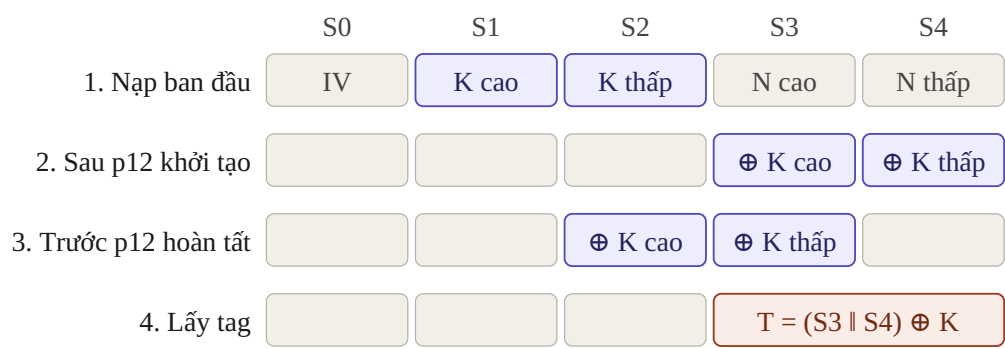
```
S2 ⊕= K[127:64]
S3 ⊕= K[63:0]

S = p12(S)

T = (S3 || S4) ⊕ K
```

Vì sao XOR khóa lần thứ hai? Đây là thứ khiến tag không giả mạo được. Không có bước này, tag sẽ chỉ là hàm của trạng thái công khai — kẻ tấn công tự tính được tag cho một gói tin bịa ra.

6.7. Vị trí XOR khóa — chỗ rất dễ nhầm



Khởi tạo XOR vào S3, S4 — hoàn tất XOR vào S2, S3. Khác nhau.

Hình 9 — Khóa chạm vào những từ trạng thái nào, ở thời điểm nào

Tóm lại phần 6

- Bốn giai đoạn: khởi tạo (p12), xử lý AD (p8), xử lý bản rõ (p8), hoàn tất (p12).
- Khóa XOR vào hai lần — đầu và cuối — nhưng ở những từ trạng thái khác nhau.
- Bit phân tách miền luôn phải XOR, kể cả khi AD rỗng.
- Không chạy p8 sau khối bản rõ cuối cùng.

Phần 7 — Giải mã và kiểm tra tag

Phần này trả lời: chiều giải mã khác chiều mã hóa ở đâu, và xử lý tag sai thế nào.

7.1. Ba giai đoạn giống hệt

Giai đoạn 1, 2, 4 **hoàn toàn không đổi**. Chỉ giai đoạn 3 khác. Và ngay cả giai đoạn 3 cũng **không cần hoán vị nghịch** — bạn không bao giờ phải chạy `p` ngược. Đây là ưu điểm thiết kế lớn: phần cứng giải mã dùng lại 100 % phần cứng mã hóa.

7.2. Giai đoạn 3 khi giải mã

Khối đầy đủ (16 byte)

$P_i = (S_0 \parallel S_1) \oplus C_i$	← tính bản rõ
$S_0 \parallel S_1 = C_i$	← GHI ĐỀ rate bằng bản mã

Khi mã hóa ta XOR bản rõ vào rồi *đọc* ra; khi giải mã ta XOR bản mã vào để *lấy* bản rõ rồi *ghi* đề rate bằng bản mã. Kết quả cuối cùng rate mang cùng giá trị `Ci` ở cả hai chiều — đó là lý do trạng thái tiến triển giống nhau.

Khối cuối chỉ có ℓ byte ($\ell < 16$) — chỗ dễ sai

Không thể ghi đề cả rate, vì phần sau byte thứ ℓ phải là **đệm**, không phải dữ liệu.

$P = \ell \text{ byte đầu của } (S_0 \parallel S_1) \oplus C$	← lấy phần bản rõ thật
$S_0 \parallel S_1 \oplus = \text{pad}(P)$	← XOR bản rõ ĐÃ ĐỆM vào, y như mã hóa

Cụ thể ở mức byte: ℓ byte đầu của rate trở thành `C`, và byte ở vị trí ℓ bị XOR thêm `0x01`.

Đừng nhớ nhầm. Câu "giải mã chỉ là ghi đề rate bằng bản mã" chỉ đúng với **khối đầy đủ**. Khối cuối lẻ byte là ngoại lệ, và là chỗ đáng viết một test riêng.

7.3. Kiểm tra tag

tính T' theo giai đoạn 4
NẾU $T' == T$: trả về <code>P</code>
NGƯỢC LẠI: trả về <code>⊥</code> và HỦY toàn bộ <code>P</code> đã tính

- Không được trả về bản rõ trước khi tag kiểm tra xong.** Nếu bạn đẩy bản rõ ra ngoài rồi mới báo tag sai, kẻ tấn công đã lấy được dữ liệu — đúng kiểu tấn công mà AEAD sinh ra để ngăn.
- Nên so tag theo kiểu constant-time.** Trên phần mềm, so sánh đừng sớm làm rò rỉ thông tin qua thời gian, cho phép dò từng byte tag. Trên phần cứng, so 128 bit song song bằng cây XOR thì tự nhiên đã constant-time — một điểm cộng đáng nêu trong báo cáo.

Tóm lại phần 7

- Giải mã dùng lại toàn bộ phần cứng mã hóa, không cần hoán vị nghịch.
- Khối đầy đủ thì ghi đè rate bằng bản mã; khối cuối lẻ byte thì XOR bản rõ đã đệm.
- Tag sai thì không được xuất một bit bản rõ nào ra ngoài.

Phần 8 — Quy tắc bảo mật khi sử dụng

Phần này trả lời: thuật toán đúng rồi thì còn phải giữ những quy tắc gì để không mất an toàn.

8.1. Nonce phải duy nhất

Cùng một khóa K , không bao giờ được dùng lại cùng một nonce N . Đây không phải khuyến nghị. Vi phạm là **mất toàn bộ bảo mật**, không phải "giảm một chút".

Vì sao: trạng thái sau giai đoạn 1 chỉ phụ thuộc (K, N) . Nếu lặp cặp này, hai thông điệp có trạng thái xuất phát giống hệt nhau. Với hai bản mã $C = P \oplus X$ và $C' = P' \oplus X$ cùng X , kẻ tấn công tính $C \oplus C' = P \oplus P'$ — khóa biến mất khỏi phương trình.

Cách tạo nonce	Đánh giá
Bộ đếm tăng dần, lưu bền vững	Tốt nhất — nhưng phải chịu được mất điện
LFSR 128 bit	Rất hợp phần cứng, rẻ hơn bộ cộng
Ngẫu nhiên 128 bit từ nguồn entropy tốt	Xác suất trùng cực nhỏ nhưng khác 0
Bộ đếm reset về 0 khi khởi động lại	Sai — mất điện là lặp nonce
Timestamp	Sai — hai gói cùng giây là lặp

8.2. Giới hạn dữ liệu

Giới hạn	Giá trị	Khi chạm tới
Tổng dữ liệu dưới một khóa	254 byte	Phải đổi khóa
Số lần giải mã thất bại dưới một khóa	2^{32}	Phải đổi khóa

Giới hạn thứ hai chống **tấn công vét cạn** tag. Với thiết bị IoT thực tế, cả hai gần như không bao giờ chạm tới — nhưng **vẫn nên viết vào báo cáo** rằng bạn biết chúng tồn tại.

8.3. Ascon không chống tấn công phát lại

Ascon đảm bảo gói tin không bị sửa và đến từ người có khóa. Nhưng nó **không** phân biệt được gói tin gốc với chính gói đó bị chặn và gửi lại.

Cảm biến gửi: "mở khóa cửa" → [kẻ tấn công ghi lại] → server: OK
Một giờ sau: kẻ tấn công phát lại y nguyên gói cũ → server: OK (!)

Gói tin hoàn toàn hợp lệ — tag đúng, vì nó là gói tin thật. Phải có cơ chế **freshness** ở tầng trên: bộ đếm tăng dần trong AD, timestamp có cửa sổ, hoặc Bloom filter lưu các nonce đã nhận.

Hướng mở rộng cho đồ án. Bloom filter là cấu trúc dữ liệu xác suất rất tiết kiệm bộ nhớ, hợp phần cứng, và có thể dùng chính Ascon-XOF128 để băm nonce sinh chỉ mục — vì XOF dùng chung lõi hoán vị với AEAD nên không cần thêm khối SHA riêng.

Tóm lại phần 8

- Nonce lặp dưới cùng một khóa là mất sạch bảo mật — dùng bộ đếm bền vững hoặc LFSR.
- Có giới hạn dữ liệu và giới hạn số lần giải mã thất bại cho mỗi khóa.
- Ascon không chống phát lại; cần cơ chế freshness ở tầng trên.

Phần 9 — Mười lỗi hay gặp nhất

Phần này trả lời: khi test vector báo sai thì dò từ đâu.

#	Lỗi	Triệu chứng
1	Nhầm Ascon-128 (v1.2) với Ascon-AEAD128	Sai từ vector đầu tiên. Kiểm: rate 128 hay 64? p8 hay p6?
2	Thứ tự byte — nạp byte vào từ 64 bit sai thứ tự	Sai mọi vector, kể cả vector đơn giản nhất
3	Quên khối đệm phụ khi dữ liệu vừa khít 16 byte	Đúng với độ dài lẻ, sai đúng ở bội số 16
4	Chạy p8 sau khối bản rõ cuối	Bản mã đúng, tag sai
5	Quên bit phân tách miền khi AD rỗng	Đúng khi có AD, sai khi AD rỗng
6	Vẫn xử lý AD khi A rỗng	Sai đúng ở các vector AD rỗng
7	Sai vị trí XOR khóa (S3,S4 vs S2,S3)	Bản mã đúng, tag sai
8	Sai khối cuối lẻ byte khi giải mã	Mã hóa đúng, giải mã sai khi độ dài không chia hết 16
9	Xoay trái thay vì phải trong p_L	Sai ngay từ một vòng đơn lẻ
10	Bảng hằng số vòng lệch index (p8 bắt đầu ở 8)	Một vòng đúng, nhiều vòng sai

Chiến lược debug hiệu quả nhất. Đừng debug ở mức thông điệp. Đi từ dưới lên: kiểm **một vòng** trước, rồi **p12 từ trạng thái toàn 0**, rồi **chỉ giai đoạn 1**, rồi mới tới thông điệp đầy đủ. Sai ở tầng nào thì sửa tầng đó. Debug thẳng ở mức test vector là cách tốn thời gian nhất.

Cái gì không cần quan tâm

Thứ	Có cần không
Ascon-CXOF128	Không — chỉ cần nếu làm phần XOF tùy biến
Ascon-80pq	Không — thuộc v1.2, không nằm trong chuẩn NIST
Các biến thể có hậu tố a	Không — thuộc v1.2
Cắt ngắn tag (truncation)	Chỉ cần biết tên — tùy chọn cắt tag xuống 64–128 bit
Che nonce (nonce masking)	Chỉ cần biết tên — tùy chọn chống lộ nonce
Ascon-Hash256 / XOF128	Tùy chọn — dùng chung lõi hoán vị nên rất rẻ để thêm

Phần 10 — Câu hỏi tự kiểm tra

Trả lời được không nhìn tài liệu thì bạn đã nắm chắc thuật toán.

Bối cảnh và đề tài

- 1. Vì sao cần mật mã hạng nhẹ thay vì dùng AES?
- 2. Họ Ascon có mấy thuật toán, chúng chia sẻ điểm gì chung?
- 3. Ascon-128 và Ascon-AEAD128 khác nhau ở những điểm nào?
- 4. Vì sao chọn APB thay vì AHB hay AXI?

Khái niệm cơ bản

- 5. AEAD đảm bảo ba tính chất gì?
- 6. Nonce khác khóa ở chỗ nào? Vì sao nonce công khai được?
- 7. Associated Data khác bản rõ ở điểm nào?
- 8. Rate quyết định điều gì, capacity quyết định điều gì?

Hoán vị

- 9. Một vòng gồm mấy lớp, tên gì, mỗi lớp làm gì?
- 10. Vì sao cần một lớp phi tuyến? Không có thì sao?
- 11. p_S trộn theo chiều nào, p_L theo chiều nào?
- 12. p_{12} và p_8 khác nhau ra sao ngoài số vòng?

Luồng và đệm

- 13. Kể tên 4 giai đoạn theo thứ tự, mỗi giai đoạn dùng p mấy vòng?
- 14. Khóa được XOR vào trạng thái mấy lần, ở đâu, vì sao?
- 15. Bit phân tách miền chống được tấn công gì?
- 16. Vì sao không chạy p_8 sau khối bản rõ cuối?
- 17. AD dài đúng 32 byte thì có mấy khối?
- 18. Bản rõ rỗng thì mấy khối? AD rỗng thì mấy khối? Vì sao khác nhau?

Giải mã và bảo mật

- 19. Giải mã có cần hoán vị nghịch không? Khác mã hóa ở đâu?
- 20. Khối cuối lẻ byte khi giải mã xử lý thế nào?
- 21. Lặp nonce dưới cùng một khóa thì hậu quả gì?
- 22. Ascon có chống được tấn công phát lại không? Vì sao?

THAM SỐ
K = 128 N = 128 T = 128 r = 128 c = 192
IV = 0x00001000808c0001
BỐN GIAI ĐOẠN
1. KHỞI TẠO S = IV K N ; p12 ; S3,S4 ⊕= K
2. AD nếu A >0: mỗi khối → rate ⊕= Ai ; p8
luôn: S4 ⊕= 0x8000000000000000
3. BẢN RÕ mỗi khối → rate ⊕= Pi ; Ci = rate ; p8 (trừ khối cuối)
4. HOÀN TẤT S2,S3 ⊕= K ; p12 ; T = (S3 S4) ⊕ K
ĐỀM
thêm 0x01 rồi bù 0x00 — số khối = ceil((len+1)/16)
AD rỗng → 0 khối Bản rõ rỗng → 1 khối
MỘT VÒNG HOÁN VỊ
p_C (S2 ⊕= const) → p_S (64 hộp thế 5 bit) → p_L (xoay + XOR)
hàng số: p12 → index 4..15 p8 → index 8..15
xoay: S0: 19,28 S1: 61,39 S2: 1,6 S3: 10,17 S4: 7,41
GIẢI MÃ
giai đoạn 1, 2, 4 y hết mã hóa — không cần hoán vị nghịch
giai đoạn 3: khối đủ → ghi đè rate = Ci
khối cuối 0B → XOR pad(P) vào rate
tag sai → trả 1, KHÔNG xuất bản rõ
GIỚI HẠN
nonce duy nhất tuyệt đối · dữ liệu ≤ 2^54 byte mỗi khóa